

Resolución de problemas y algoritmos

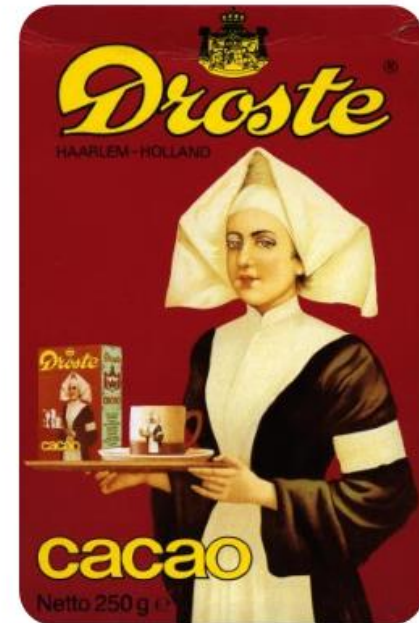
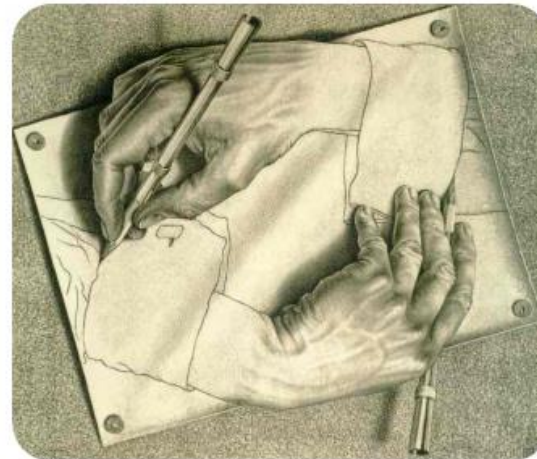
Dra. Jessica Andrea Carballido



Dpto. de Ciencias e Ingeniería de la Computación
UNIVERSIDAD NACIONAL DEL SUR



Imágenes recursivas



Para entender recursividad,
primero hay que entender
qué es la recursividad.

Un programador anónimo



**ALTERNATIVA AL USO
DE CICLOS**

RECURSIVIDAD



No sé subir
una escalera completa.
Sé SUBIR 1 ESCALON.

RECURSIVIDAD



Problema. Subir una escalera de N escalones

Caso Trivial o Base: Si la escalera tiene un escalón,
SUBIR 1 ESCALON.

Caso General o Recursivo: Si la escalera tiene más de un escalón,
para Subir una escalera de N escalones podemos **SUBIR 1 ESCALON**
y luego Subir una escalera de $(N-1)$ escalones.



INSTANCIA
MENOR

RECURSIVIDAD



CASO BASE: Instancia para la que tenemos una solución directa.

CASO RECURSIVO: Instancia en la que usamos la solución de una instancia *menor o más simple* del mismo problema, para resolver la instancia actual.

Una instancia *menor* siempre está mas cerca d que la instancia actual.

WHAT ARE YOU WORKING ON?

TRYING TO FIX THE PROBLEMS I CREATED WHEN I TRIED TO FIX THE PROBLEMS I CREATED WHEN I TRIED TO FIX THE PROBLEMS I CREATED WHEN...



RECURSIVIDAD



La recursividad es un modo de definir *algo* en términos de sí mismo.

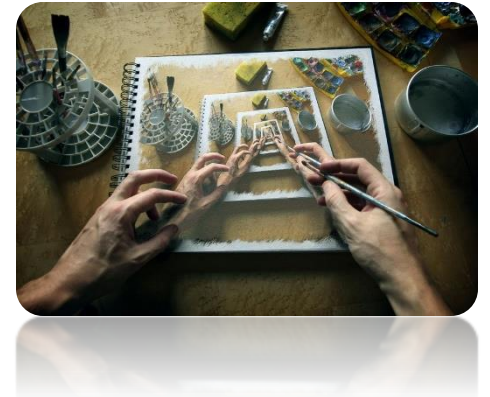
En apariencia, esta definición es circular.

Con más precisión, **las instancias más complejas se especifican a partir de instancias más simples y las más simples se especifican directamente.**

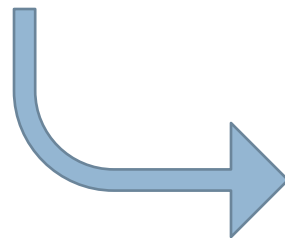
Ese *algo* puede ser un dibujo, un operación, un algoritmo, un proceso, etc.

La recursividad aparece entonces en el Lenguaje, en la Pintura, en la Música y en la Naturaleza.

RECURSIVIDAD



¿Y qué es una
instancia?

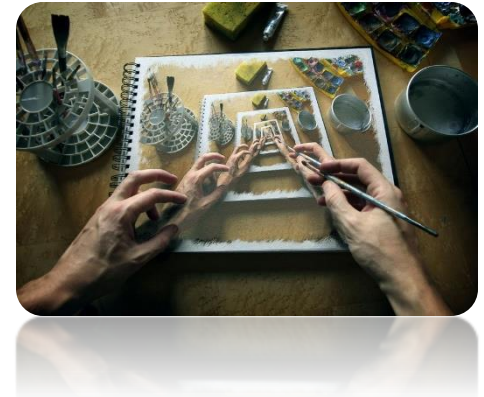


UNA INSTANCIA ES UN **CASO PARTICULAR** QUE ESTAMOS RESOLVIENDO EN UN MOMENTO DADO (no es exclusivo de la recursividad!)

Recursividad en Matemática

PLANTEO

$$\text{base}^{\text{exp}} \left\{ \begin{array}{ll} | & \text{exp} = 0 \\ \text{base} \times \text{base}^{\text{exp}-1} & \text{exp} > 0 \end{array} \right.$$



$$\begin{aligned} 2^4 &= 2 \times 2^3 = 2 \times (2 \times 2^2) = 2 \times (2 \times (2 \times 2^1)) \\ &= 2 \times (2 \times (2 \times (2 \times 2^0))) \\ &= 2 \times (2 \times (2 \times (2 \times 1))) \\ &= 2 \times (2 \times (2 \times 2)) \\ &= 2 \times (2 \times 4) \\ &= 2 \times 8 = 16 \end{aligned}$$

Potencia

RECURSIVIDAD



La recursividad resulta adecuada en problemas que reúnen las siguientes propiedades:

- ✓ Existe al menos un caso trivial, que es el que puede resolverse directamente.
- ✓ Resolver cualquier instancia del problema - excepto el caso trivial - requiere resolver una o varias instancias más simples del mismo problema.

Una instancia es más simple que otra si está más cerca del caso trivial.

RECURSIVIDAD

Potencia



Implementar en Pascal la siguiente definición recursiva:

Caso trivial o caso base

$\text{exp} = 0$

Se asume
 exp mayor o igual a 0.

base^{exp}

$\text{base} \times \text{base}^{\text{exp}-1}$

$\text{exp} > 0$

Caso general

$\text{Potencia}(2,4) = \text{Potencia}(2,3) * 2$

$\text{Potencia}(2,3) = \text{Potencia}(2,2) * 2$

$\text{Potencia}(2,2) = \text{Potencia}(2,1) * 2$

$\text{Potencia}(2,1) = \text{Potencia}(2,0) * 2$

$\text{Potencia}(2,0) = 1$

$\text{Potencia}(2,4) = 8 * 2 = 16$

$\text{Potencia}(2,3) = 4 * 2 = 8$

$\text{Potencia}(2,2) = 2 * 2 = 4$

$\text{Potencia}(2,1) = 1 * 2 = 2$

RECURSIVIDAD



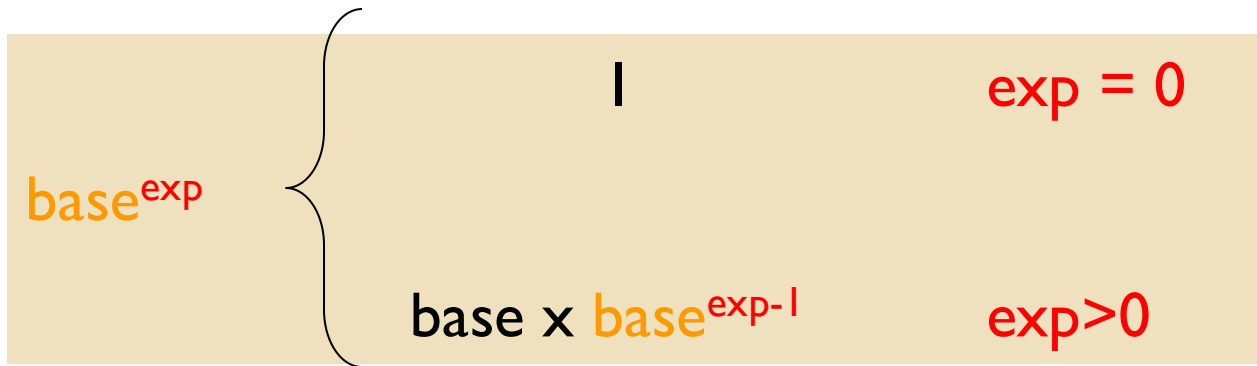
El diseño de un algoritmo recursivo
consiste en identificar:

- ✓ Caso base o trivial: Cuándo estamos en el caso trivial, y cual es la solución para el problema en ese caso.
- ✓ Caso recursivo o general: Cuándo estamos en el caso general, y cómo resolvemos el caso general a partir de la solución de una instancia menor.

Cuando el problema está correctamente **PLANTEADO**
de manera recursiva,
la implementación es directa.

RECURSIVIDAD

Implementar en Pascal la siguiente definición recursiva:



```
function Potencia (base, exp: integer): integer;  
begin  
  if (exp = 0)  
    then Potencia := 1  
    else Potencia := base * Potencia(base, exp-1);  
end;
```

RECURSIVIDAD

Program PrimerRECURSIVO;

```
function Potencia(base, exp: integer): integer;
```

```
begin
```

```
if exp = 0
```

```
    then Potencia := 1
```

Caso trivial o caso base

```
    else
```

```
        Potencia := base * Potencia(base, exp-1);
```

Caso general

```
end;
```

```
begin
```

```
Write(Potencia(3,2));
```

```
end;.
```



3

2

```

function Potencia (m, n: integer): integer;
begin
  if n=0 then Potencia:=1
    else Potencia:=m*Potencia(m, n-1);
end;

```



(3, 1)

3

1

```

function Potencia (m, n: integer): integer;
begin
  if n=0 then Potencia:=1
    else Potencia:=m*Potencia(m, n-1);
end;

```

(3, 0)

3

0

```

function Potencia (m, n: integer):
begin
  if n=0 then Potencia:=1
    else Potencia:=m*Potencia
end;

```

m. Base
n. Exponente

...

```

Begin
Write(Potencia(3,2);
End.

```

3

2

```

function Potencia (m, n: integer): integer;
begin
  if n=0 then Potencia:=1
    else Potencia:=m*Potencia(m, n-1);
end;

```



(3, 1)

3

1

```

function Potencia (m, n: integer): integer;
begin
  if n=0 then Potencia:=1
    else Potencia:=m*Potencia(m, n-1);
end;

```

(3, 0)

1

m. Base
n. Exponente

...

```

Begin
Write(Potencia(3,2);
End.

```

3

2

```
function Potencia (m, n: integer): integer;  
begin  
  if n=0 then Potencia:=1  
    else Potencia:=m*Potencia(m, n-1);  
end;
```

3

(3, 1)

m. Base
n. Exponente



...

```
Begin  
Write(Potencia(3,2));  
End.
```

RECURSIVIDAD



En Ciencias de la Computación,
para que un algoritmo sea recursivo
es **condición necesaria** (pero no suficiente)
que se invoque a sí mismo.

La recursividad permite resolver un problema
a partir de **reducirlo a *instancias más simples***
de sí mismo.

La reducción depende del tipo de problema
que estemos resolviendo.

EXPLAINING RECURSION



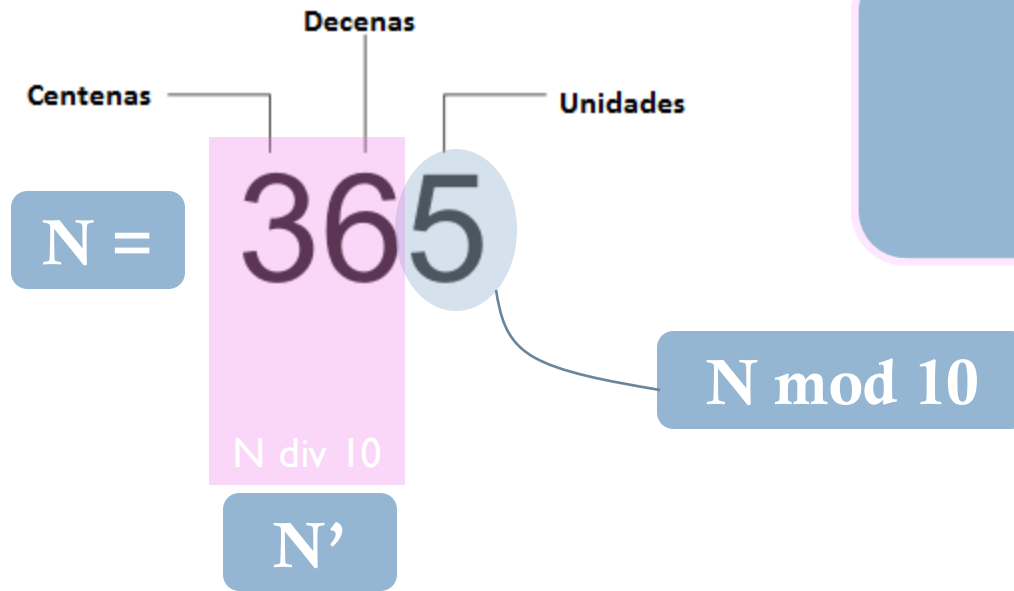
TO NON PROGRAMMERS



A two-panel meme featuring Spider-Man. In the first panel, Spider-Man is standing next to a white van with 'NYPD' written on its side, looking towards the camera. In the second panel, Spider-Man is seen from behind, pointing his right index finger at a red wall. The word 'RECURSION' is overlaid in large, white, bold, sans-serif capital letters with a black outline on both panels.

RECURSION

RECURSION



LA REDUCCIÓN
ES SACANDO
EL último dígito

**Recorrido
de números
con recursividad**

RECURSIVIDAD



Problema: Calcular la cantidad de dígitos de un número natural N (mayor que cero).

$$\text{cantDigitos}(N) = \begin{cases} 1 & \text{si } N < 10 \\ \text{cantDigitos}(N \text{ div } 10) + 1 & \text{si } N \geq 10 \end{cases}$$

Cantidad de Dígitos de N

Caso trivial: Si $N < 10$, la Cantidad de Dígitos de N es 1.

Caso recursivo: Si $N \geq 10$, la Cantidad de Dígitos de N es igual a la Cantidad de Dígitos de N' más 1.

Donde N' es N sin su dígito menos significativo.

```
program testCuantosDigitos;  
var numero: integer;
```

Instancia actual



```
function cantDigitos(N: integer): integer;  
begin
```

```
  if N < 10
```

```
  then
```

```
    cantDigitos := 1
```

```
  else
```

```
    cantDigitos := cantDigitos(N div 10) + 1;
```

```
end;
```

```
begin
```

```
  write('Ingrese un número natural:');
```

```
  readln(numero);
```

```
  writeln('La cantidad de dígitos de ', numero, ' es ', cantDigitos(numero));
```

```
end.
```

Resultado de la instancia menor (el resultado de invocar a la función)

Instancia menor (N sin su último dígito)

Podemos asumir que el usuario
ingresa un número natural, sin validarlo.

Primitiva
leerNatural(numero)

RECURSIVIDAD



Problema: Calcular la suma de los dígitos de un número natural N .

Si $N = d_m d_{m-1} \dots d_1 d_0$ la función $\text{sumaDigitos}(N)$ computa:

$$d_m + d_{m-1} + \dots + d_1 + d_0$$

$$\text{sumaDigitos}(N) = \begin{cases} N & \text{si } N < 10 \\ \text{sumaDigitos}(N \text{ div } 10) + N \bmod 10 & \text{si } N \geq 10 \end{cases}$$

SumaDigitos de N

Caso trivial: Si $N < 10$, la SumaDigitos de N es N .

Caso recursivo: Si $N \geq 10$, la SumaDigitos de N es igual a la suma de los dígitos de N' más el dígito menos significativo de N . Donde N' es N sin su dígito menos significativo.

```
program testSuma;
  var numero: integer;

  function sumaDigitos(N: integer): integer;
  begin
    if N < 10
    then
      sumaDigitos := N
    else
      sumaDigitos := sumaDigitos(N div 10) + N mod 10;
    end;

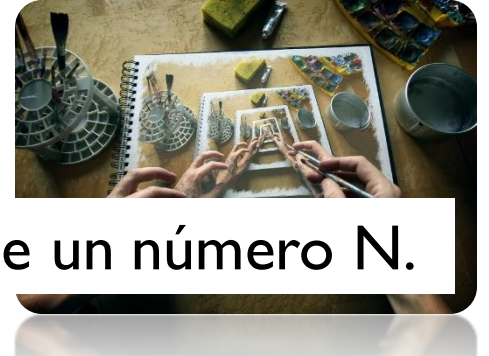
  begin
    write('Ingrese un número natural:');
    readln(numero);
    writeln('La suma de los dígitos de ', numero, ' es ', sumaDigitos(numero));
  end.
```



Podemos asumir que el usuario
ingresa un número natural, sin validarlo.

Primitiva
leerNatural(numero)

RECURSIVIDAD



Problema: Calcular la suma de los **dígitos pares** de un número N .

Buscamos ejemplos que nos permitan identificar el o los **casos triviales**.

La suma de los dígitos pares de 6 es 6.

La suma de los dígitos pares de 9 es 0.

La suma de los dígitos pares de 5 es 0.

Podemos generalizar en dos casos triviales.

RECURSIVIDAD



Problema: Calcular la suma de los **dígitos pares** de un número N .

Buscamos ejemplos que nos permitan identificar el o los **casos recursivos**.

La suma de los dígitos pares de 662 es la suma de los dígitos pares de 66 más 2.

La suma de los dígitos pares de 66324 es la suma de los dígitos pares de 6632 más 4.

La suma de los dígitos pares de 66325 es la suma de los dígitos pares de 6632.

Podemos generalizar en dos casos recursivos.

RECURSIVIDAD



PLANTEO Suma de dígitos pares de N

Caso trivial 1: Si $N < 10$, y es impar, la Suma de dígitos pares de N es 0.

Caso trivial 2: Si $N < 10$, y es par, la Suma de dígitos pares de N es N.

2 casos
triviales

Caso General 1: Si $N \geq 10$, y el dígito menos significativo de N es par, la Suma de dígitos pares de N es igual a la Suma de dígitos pares de N' más el dígito menos significativo de N.

Caso General 2: Si $N \geq 10$, y el dígito menos significativo de N es impar, la Suma de dígitos pares de N es igual a la Suma de dígitos pares de N'.

Donde **N'** es **N** sin su dígito menos significativo.

RECURSIVIDAD

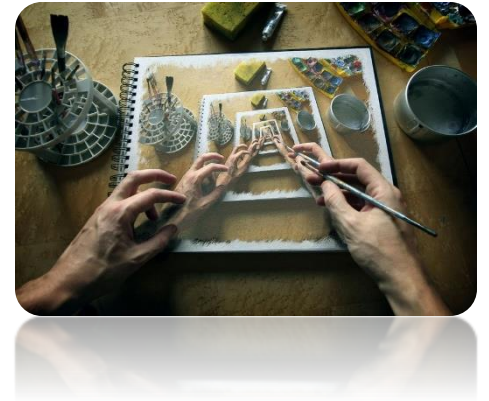


```
function sumaDP(N: integer): integer;  
begin  
  if (N < 10)  
  then  
    if (odd(N))      { impar(N) }  
    then sumaDP := 0  
    else sumaDP := N  
  else  
    if ((N mod 10) mod 2 = 0)  
    then  
      sumaDP := sumaDP(N div 10) + N mod 10  
    else  
      sumaDP := sumaDP(N div 10);  
end;
```

Función que **respeta**
el **planteo** propuesto

el **planteo** propuesto

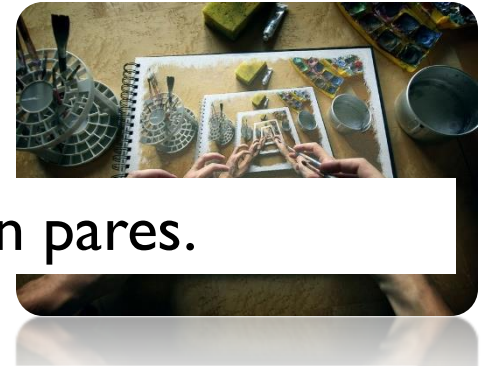
VER VIDEO (13 min.)



VIDEO JESSICA EXPLICANDO PROBLEMA RECURSIVO

<https://youtu.be/7VHzYsiYkJc?si=iTrRzOJu6Q2icJE5>

RECURSIVIDAD



Problema: Determinar si todos los dígitos de N son pares.

Planteo: N tiene todos los dígitos pares (TDP)?

CBI: Si $N \leq 9$, y es par, N tiene TDP.

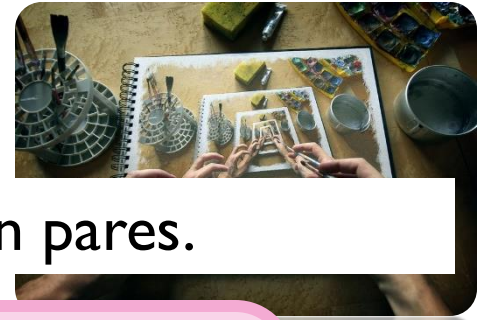
CB2: Si $N \leq 9$, y es impar, N **no** tiene TDP.

CG: Si $N > 9$, N tiene TDP si N' tiene TDP y además, el último dígito de N es par. En cualquier otro caso, N **no** tiene TDP.

Donde N' es N sin su último dígito.

Sé cuándo UN DÍGITO
es PAR.

RECURSIVIDAD



Problema: Determinar si todos los dígitos de N son pares.

```
function TDP(N: integer): boolean;  
var ultimo: integer;  
begin  
  if (N<=9)  
    then  
      if (N mod 2=0) then TDP:=TRUE  
      else TDP:=FALSE  
    else  
      begin  
        ultimo:=N mod 10;  
        TDP:= TDP(N div 10) and (ultimo mod 2=0);  
      end;  
    end;  
end;
```



Se podrá
optimizar?

RECURSIVIDAD



Problema: Determinar si todos los dígitos de N son pares.

Planteo: N tiene todos los dígitos pares (TDP)?

CB1: Si $N \leq 9$, y es par, N tiene TDP.

CB2: Si $N \leq 9$, y es impar, N **no** tiene TDP.

CB3: Si $N > 9$ y su último dígito es impar, N no tiene TDP.

CG: Si $N > 9$, y su último dígito es par, N tiene TDP **si N' tiene TDP.**
Donde N' es N sin su último dígito.

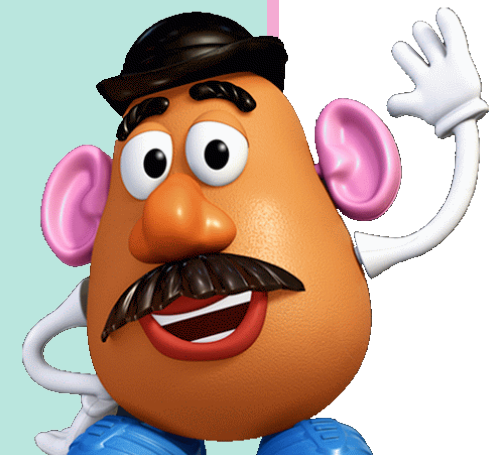
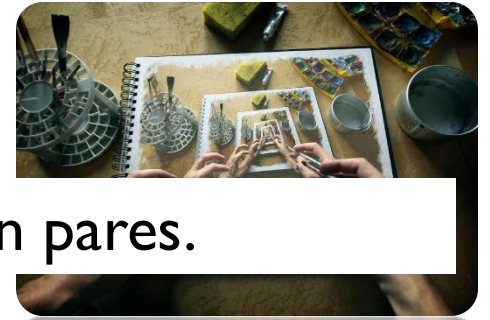
**Tres casos
triviales**

Un único caso general (recursivo)

RECURSIVIDAD

Problema: Determinar si todos los dígitos de N son pares.

```
function TDP(N: integer): boolean;  
var ultimo: integer;  
begin  
  if (N <= 9)  
    then  
      if (N mod 2 = 0) then TDP := TRUE  
      else TDP := FALSE  
    else  
      if odd(N mod 10)  
        then TDP := FALSE  
        else TDP := TDP(N div 10);  
  end;
```



**Tres casos
triviales**

Problema 1: Determinar si un dígito D aparece en un número N .

Problema 2: Determinar si todos los dígitos de N son múltiplos de 3.

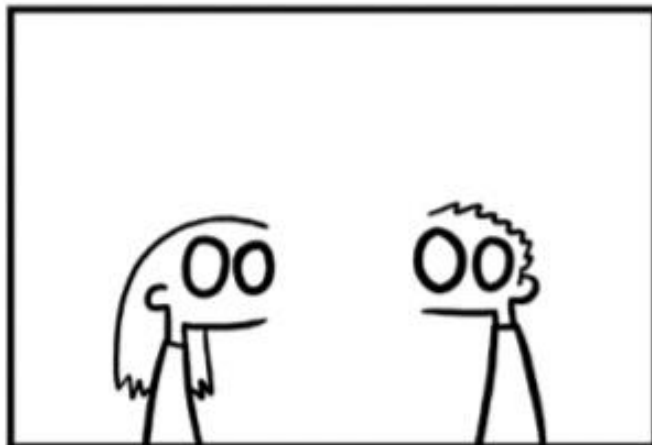
Problema 3: Calcular la cantidad de veces que aparece el dígito D en un número N .



Recursión MUTUA

```
9  program Recursion_MUTUA;
10  VAR X: INTEGER;
11
12  function esImpar(n: integer): boolean; forward;
13
14  function esPar(n: integer): boolean;
15  begin
16  if (n=0)
17  then esPar := TRUE
18  else esPar := esImpar(n-1);
19  end;
20
21  function esImpar(n: integer): boolean;
22  begin
23  if (n=0)
24  then esImpar := FALSE
25  else esImpar := esPar(n-1);
26  end;
27
28  begin
29  write('Ingresa x: ');
30  readln(X);
31  if esPar(X)
32  then write(X, ' es par. ')
33  else write(X, ' no es par. ')
34  end.
```

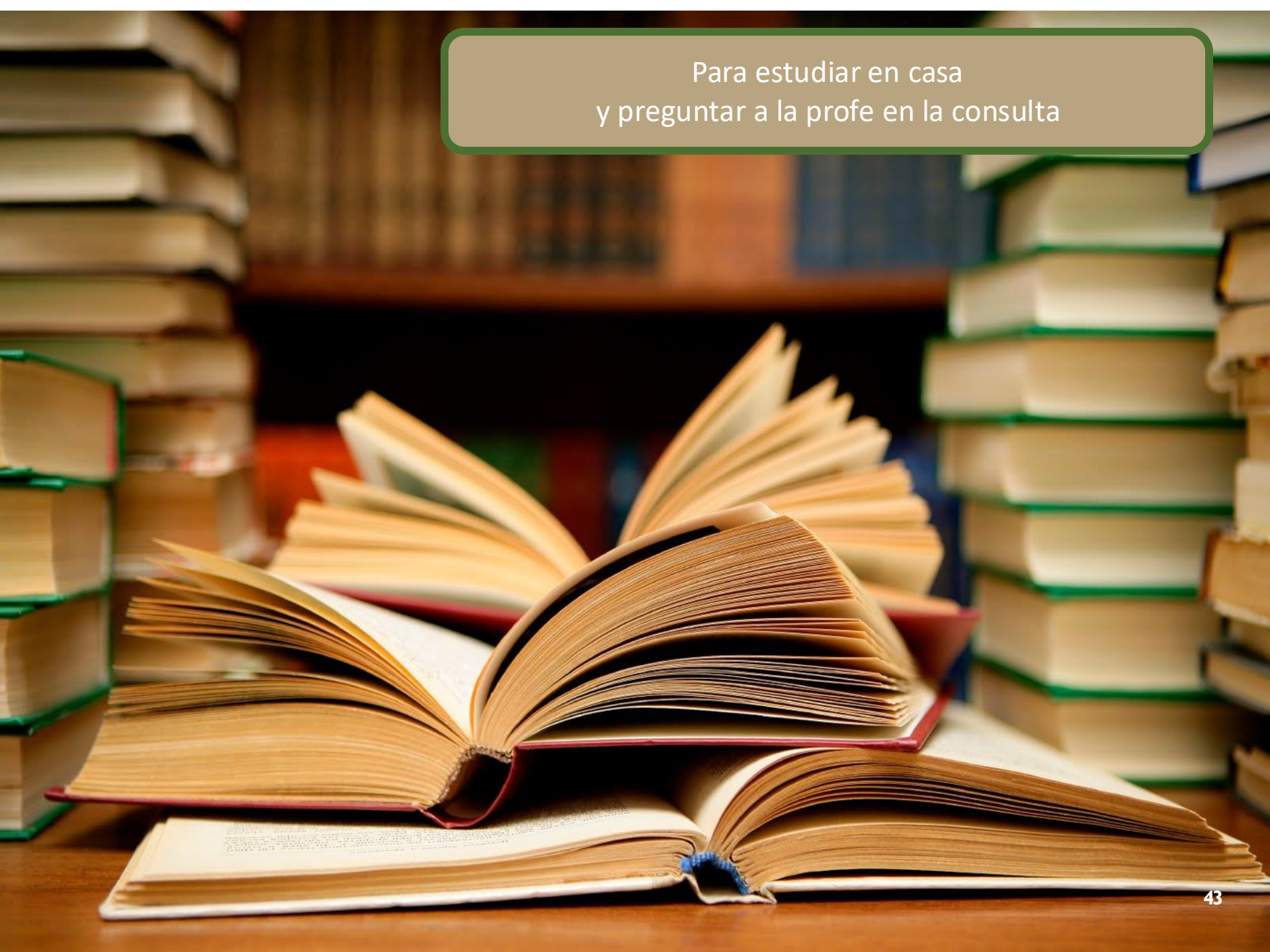




Recursión MUTUA

RECURSiViDAD

Para estudiar en casa
y preguntar a la profe en la consulta



Recursividad en Matemática

FACTORIAL



DEFINICION: El factorial de un número (tiene que ser mayor a cero) es el producto de todos los enteros entre 1 y ese número.

- Para entender esta operación, vemos un par de ejemplos:
Factorial de 10: 10!

Se calcula como $10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 3628800$

Factorial de 3: 3!

Se calcula como $3 \times 2 \times 1 = 6$

$$n! \begin{cases} 1 & \text{si } n = 1 \\ n \times (n-1)! & \text{si } n > 1 \end{cases}$$

**Definición
matemática**



Recursividad en Matemática

FACTORIAL



DEFINICION: El factorial de un número (tiene que ser mayor a cero) es el producto de todos los enteros entre 1 y ese número.

Factorial de 10: 10!

Se calcula como $10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 3628800$



Es el factorial de 9!



n	<u>n!</u>
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800
11	39,916,800
12	479,001,600
13	6,227,020,800
14	87,178,291,200
15	1,307,674,368,000
16	20,922,789,888,000



Factorial

Algunos ejemplos más...

El factorial de un número se calcula como ese número multiplicado por el factorial del número -1.

El factorial de 1 es
DIRECTAMENTE 1.

Recursividad en Matemática

PLANTEO

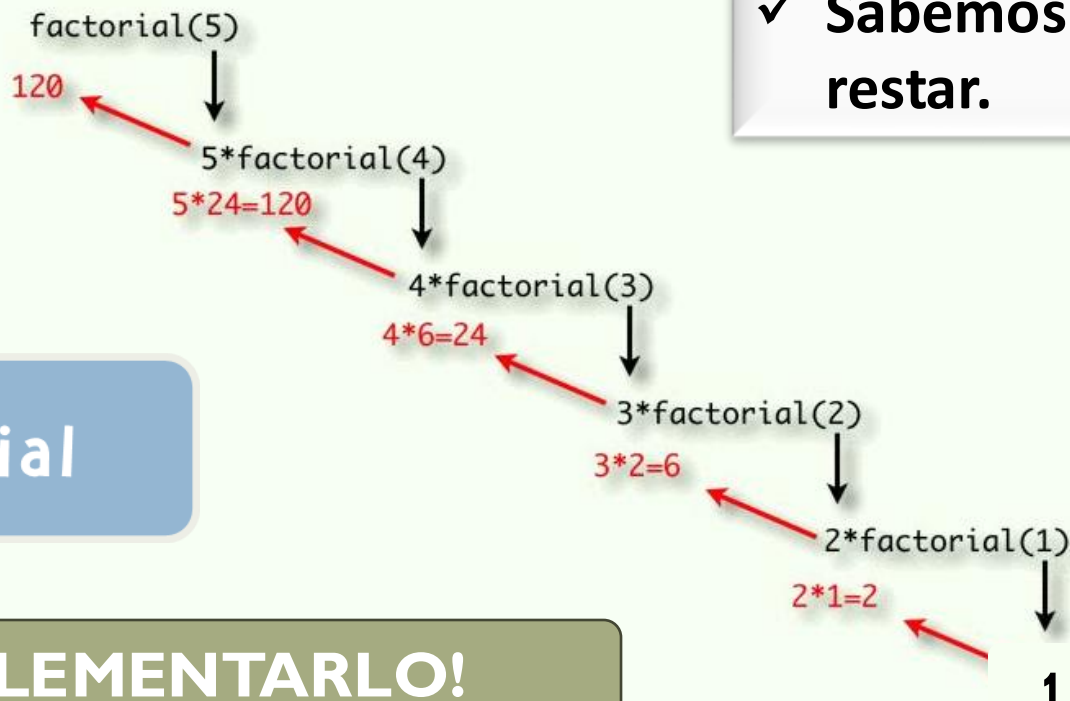
$$n! = \begin{cases} 1 & \text{si } n = 1 \\ n \times (n-1)! & \text{si } n > 1 \end{cases}$$



- ✓ Sabemos que el factorial de 1 es 1.
- ✓ Sabemos multiplicar y restar.

Factorial

¡IMPLEMENTARLO!



Recursividad



Problema: Hallar el dígito más significativo de un número natural N . Si $N = d_m d_{m-1} \dots d_1 d_0$ el dígito más significativo es d_m .

Ejemplo (**RAZONAMIENTO**):

Si N es 2456, N' es 245 (N sin su último dígito).

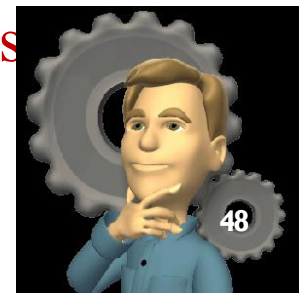
Cuál es el dígito más significativo de N' $\rightarrow$ 2

Sabiendo esto, pensamos... cuál será el dígito más significativo de N ?? Y vemos que el dígito más significativo de N es igual al dígito más significativo de N' ! Tenemos nuestro caso general.

Y el caso base?

Cual es el dígito más significativo del 6?? El 6! y del 2? el 2!

Luego, si N tiene sólo un dígito, ese dígito es el dígito más significativo de N .



Recursividad



Problema: Hallar el dígito más significativo de un número natural N . Si $N = d_m d_{m-1} \dots d_1 d_0$ el dígito más significativo es d_m .

$$\text{DigMasSig}(n) = \begin{cases} n & \text{si } n < 10 \\ \text{DigMasSig}(n \text{ div } 10) & \text{si } n \geq 10 \end{cases}$$

DigMasSig de N

Caso trivial: Si $N < 10$, el DigMasSig de N es N .

Caso general: Si $N \geq 10$, el DigMasSig de N es igual al DigMasSig de N' , donde N' es N sin su último dígito.

Recursividad



Función que
respeto el
planteo

bj9u460

```
program testNumero;  
  var numero: integer;  
  
  function DigMasSig(n: integer): integer;  
  begin  
    if (n < 10)  
    then  
      DigMasSig := n  
    else  
      DigMasSig := DigMasSig(n div 10);  
    end;  
  
  begin  
    write('Ingrese un número natural:');  
    readln(numero);  
    writeln('El digito mas significativo de ', numero, ' es ', DigMasSig(numero));  
  end.
```

DigMasSig se invoca una vez desde el programa principal
y luego recursivamente desde DigMasSig

Recursividad

Se desea calcular la suma de los primeros N números naturales. Escribir un planteo recursivo y una función que se corresponda con el planteo, para resolver este problema.

Ejemplos:

Si $N=5$, calcular $5+4+3+2+1$

Si $N=0$, la suma es 0

Si $N=10$, calcular $10+9+8+7+6+5+4+3+2+1$

Si $N=5$, calcular $5 + \quad \mathbf{4+3+2+1}$

Podemos decir que la suma de los primeros 5 naturales es igual a $5 +$ la suma de los primeros 4 naturales???

Y la suma de los primeros 4 naturales, será igual a $4 +$ la suma de los primeros 3 naturales?

Recursividad

Se desea calcular la suma de los primeros N números naturales. Escribir un planteo recursivo y una función que se corresponda con el planteo, para resolver este problema.

La suma de los primeros N naturales

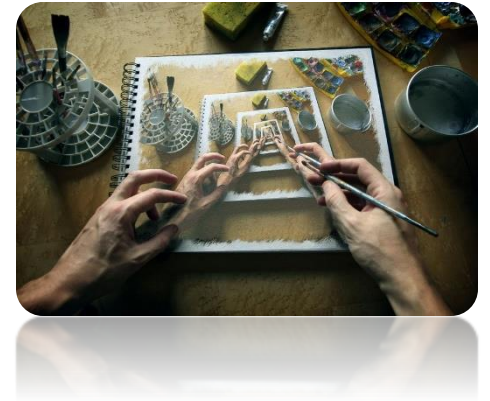
CB: Si $N=0$, la suma de los primeros N es 0.

CG: Si $N>0$, la suma de los primeros N enteros es igual a $N + \text{la suma de los primeros } (N-1) \text{ naturales}$.

```
function suma(N: integer): Integer;  
begin  
  if N=0 then suma:=0  
  else  
    suma:=N + suma(N-1);  
  end;
```

OJO
Aquí LA Reducción
ES RESTANDO 1 A N

Recursividad en Matemática



PLANTEO

$\text{MCD}(M, N)$

Asumimos $M > N$

N

$\text{MCD}(N, R)$

si N divide a M

en cualquier otro caso,
donde R es el Resto de
dividir M y N

$\text{MCD}(18, 12)$

$M \ N$

$= \text{MCD}(12, 6)$

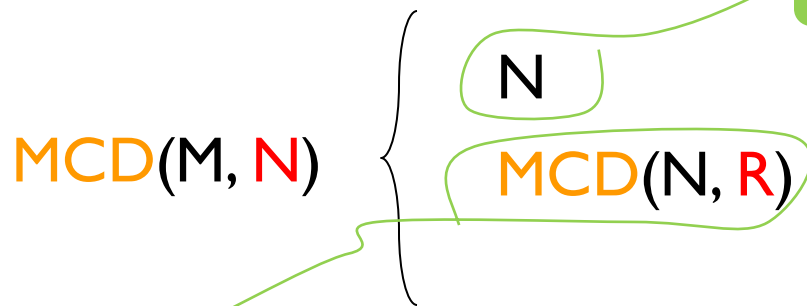
$M \ N$

$= 6$



Recursividad

Implementar en Pascal la siguiente definición recursiva:



Caso trivial

si N divide a M

en cualquier otro caso
donde R es el Resto de
dividir M por N

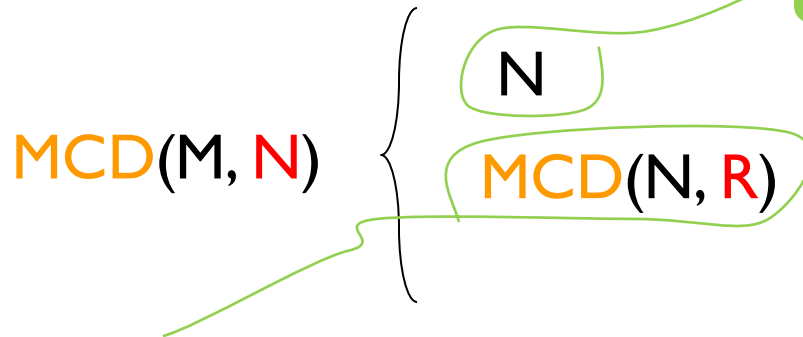
Caso general

$$MCD(10,7)=MCD(7,3)=MCD(3,1)=1$$
$$MCD(12,10)=MCD(10,2)=2$$

Sé hacer una división entera y sé
que el MCD de M y N es N
cuando N divide a M .

Recursividad

Implementar en Pascal la siguiente definición recursiva:



Caso trivial

si N divide a M

en cualquier otro caso
donde R es el Resto
de dividir M por N

Caso general

```
function MCD (M, N: integer): integer;  
begin  
  if (M mod N = 0)  
  then MCD := N  
  else MCD := MCD(N, M mod N);  
end;
```

REDUCCIÓN

No sé calcular el MCD, pero sé
hacer una división entera y sé
que el MCD de M y N es N
cuando N divide a M .